

В лекции «The Future of Programming» Брет Виктор парадоксальным образом описывает не столько наше настоящее, сколько альтернативное видение будущего, которое уже существовало в виде прототипов на заре компьютерной эры. Центральный тезис лекции можно выразить цитатой, которая служит предостережением для всех создателей: “Самая опасная мысль, которая может посетить творческого человека - будто он знает, что делает”. Это утверждение Виктор иллюстрирует тем, что технологии развиваются экспоненциально, в то время как мышление программистов цепляется за устоявшиеся практики. Именно уверенность в своей правоте порождает циклы сопротивления новому: программисты на бинарном коде не видели ценности в “расточительном” ассемблере, а приверженцы ассемблера точно так же отвергали Fortran. Виктор выделяет четыре ключевых направления, по которым должно было пойти развитие, но не пошло в силу этой самоуверенности.

Первое - переход от кодирования к прямому манипулированию данными. Пример Sketchpad демонстрирует работу с геометрическими ограничениями, что позволяет изменять объекты, не перерисовывая их. Эволюция от процедур к целям и ограничениям: так, языки Planner и Prolog предлагали программисту описывать желаемый результат, а не пошаговый алгоритм. Также и с заменой текстового вывода на представления. Разработки в NLS, системе GRAIL или браузере Smalltalk предвещали отказ от текста. И последнее - движение от последовательных к параллельным вычислениям, где модель акторов предлагала альтернативу ненадежным блокировкам и потокам. Все эти идеи были похоронены уверенностью современников в том, что они и так “знают, что делают”.

Рассматривая эти идеи из 1973 года, поражаешься не только их пророческой силе, но и точности описания причин их невнедрения. Полезность этого исторического анализа колоссальна. Лектор объясняет, почему мы до сих пор пишем код строками текста, хотя принципы прямого манипулирования давно победили в интерфейсах пользователя, но почти не проникли к программистам. Мы создаем инструменты под себя, а не под задачу, и консерватизм сообщества оказывается сильнее технического прогресса. Прямое манипулирование (как в Sketchpad) требует перестройки всей культуры разработки, а текст позволяет сохранить иллюзию полного контроля и привычной эффективности. Актуальность идей Виктора не вызывает сомнений: регулярные выражения стали стандартом, а необходимость параллельного программирования сегодня переросла в потребность из-за роста многоядерных процессоров. Однако достоверность прогнозов относительна. Виктор прав в том, что эти инструменты существовали, но он недооценил живучесть текста как универсального и гибкого инструмента. Текст оказался достаточным для передачи сложной логики, а пространственные интерфейсы часто остаются либо слишком узкоспециализированными, либо громоздкими.

Главная проблема, поднятая Виктором, не технический прогресс, а временной промежуток между изобретением и его принятием. Мы наблюдаем циклическую природу инноваций и сопротивления: каждое новое поколение программистов отбрасывает инструменты предыдущего как излишне сложные или недостаточно производительные, но с трудом принимает парадигмы, ломающие их картину мира. Скепсис в отношении ассемблера или Fortran — это та же история, что и современные споры о пользе low-code платформ или ИИ-ассистентов.

Мне кажется, что идеи Виктора о вычислительной среде, реагирующей на цели (goals) и прямую манипуляцию (direct manipulation), все еще ждут своего часа. Современные IDE только начинают двигаться в сторону семантического редактирования кода, а не просто текста. Возможно, мы стоим на пороге того самого будущего, где описание задачи станет важнее инструкций для машины, но, как и 50 лет назад, главным тормозом остается восприятие нашего собственного мышления, воспитанного на последовательных текстовых командах. Переход к по-настоящему конкурентному и интерактивному программированию потребует не просто новых языков, а когнитивной пластичности и быстрой адаптации к этому.